ACS-3922 Fall 2026

Assignment 1

Due Date: Monday, October 19th at 11:59 pm

PART A

1. Complete Tutorial 1 with your choice of assets.
    - Follow the instructions in the README file for a new project or for using an existing project.
    - You must add your own assets (i.e. replace the assets from the tutorial, by finding new assets online).
        - Include a new paddle, ball, and sound.
    - Rename the MainScene as `PartAScene`.

    → *Commit the files to your repository.*

2. Rename the paddle objects as `PlayerPaddle` and `ComputerPaddle`. You will edit this 2-player game to now be a 1-player game against the computer.
    a. Remove the `PlayerControl` script from the `ComputerPaddle` object.
    b. Create a new script called `OpponentBehaviour`. The `ComputerPaddle` should track/get the `Ball`'s `y` position and follow/move accordingly.
        - Use the `FixedUpdate()` method instead of `Update()`
            - *NOTE: This forces updates at a fixed interval (by default 50 times a second) rather than every frame, which allows logic to run at a consistent rate regardless of frame rate fluctuations.*
        - The speed of movement should be slower than the player's. Tweak the speed value so that the player should be able to get the ball past the computer's paddle while maintaining a challenging level.
            - *NOTE: Another way to tweak the ability in favour of the player is to only move the computer paddle if/when the `Ball` is on the computer's half of the game scene (i.e., the `x` position is of a certain value). Thus, the computer paddle is effectively given less time to react. You can also add some randomness to the computer's paddle to further tweak the ability. Not all of these are necessary, however may be interesting to explore as you create a game design that offers gameplay you desire.*

    → *Commit the files to your repository.*

3. Update the `BallBehaviour` script so that the player is able to have *some* control over the ball's outgoing angle when it hits the paddle. Work with the script so that if the ball hits the paddle closer to the top and bottom edges, the ball is fired back wider than if it hits the paddle near the middle.
    - *HINT: when a collision occurs (i.e., we know the ball hit a paddle), evaluate the difference in the `Ball`'s `y` position and the `PlayerPaddle`'s `y` position. A small or zero difference will mean the ball and paddle are at the same `y` location and thus the ball hit the center of the paddle. A larger difference suggests the ball hit the paddle further from the center.*

   → *Commit the files to your repository.*

4. Change the `StartBall()` method in the `BallBehaviour` script so that the ball's first direction is more random. Make sure that all possibilities are reasonable and that the ball travels at a good speed.

   → *Commit the files to your repository.*

5. Change the UI messaging to reflect that it is only a 1-player game and if the computer wins that they have done so rather than Player 2.

   → *Commit the files to your repository.*

[BONUS – THIS DOES NOT NEED TO BE COMPLETED FOR THE CHANCE TO RECEIVE FULL MARKS ON THE ASSIGNMENT]
Include additional sound assets, such that there is one for when the Ball collides with the upper and lower walls, one when a player scores a point, and one when the Ball collides with a paddle.

→ *Push your project code to the remote repository to make a submission. You can push as much as you would like, however it is often best to do it when a complete solution to a part is present. Pushing after the deadline will constitute a late submission.*

PART B

Note that no further Git/Github instructions will be provided. Be sure to commit regularly and push at the end to submit your code as a submission for the assignment.

1. Create a new scene and name it `PartBScene`. Note that you will add assets to this game as well to give it your own personal look and feel.

Build a brick game as shown here. You must use a Prefab in your solution and instantiate the bricks at runtime. While not shown in the video, a message should also display when the player loses their last life.

2. Add a new scene named `Menu`.
    a) The screen must include:
        - An image of your choice
        - Buttons to start each game and to quit the application.
        - Optional keyboard input for each selection (e.g., P, B, or Q)
        - Your own original names for each game (included in the button options)
    b) Edit the end of both games so that the user will return to the Main Menu when they press the escape key.

PART C

Create a new markdown file called `A1PartC.md`. You must use at least **two of: headings, external links, bullets, and bold/italics** within the markdown file.

1. Add you name and student number to the document.

2. Add the sources that you used for your game and any additional notes that you may have for your marker.

3. Discuss for each of Part A and Part B your programming choices which led to changes in difficulty within the game (couple of succinct, roughly 5 sentence, paragraphs for each part). Relate choices to specific *Design Goals* discussed in the slides and how these choices impact specific goals. It is likely the case that simple games such as Pong and Breakout do not play as strongly to all goals. However, you could still discuss aspects of all or some of:
    - Fun/Engaging
    - Flow
    - Empowerment
    - Attention and Involvement
    - Decisions
    - Experiential Understanding